NTNU
Kunnskap for en bedre verden

Institutt for datateknologi og informatikk

TDT4145 – Datamodellering og databaser

Prosjektrapport

Forfattere:
Axel Filseth, Leo Egilson Skarbø, Knut Opland Moen

March 4, 2026

Contents

List of Figures

List of Tables

1 ER-diagram

ER-diagrammet vist i **??** viser entiteter, relasjoner og attributter.

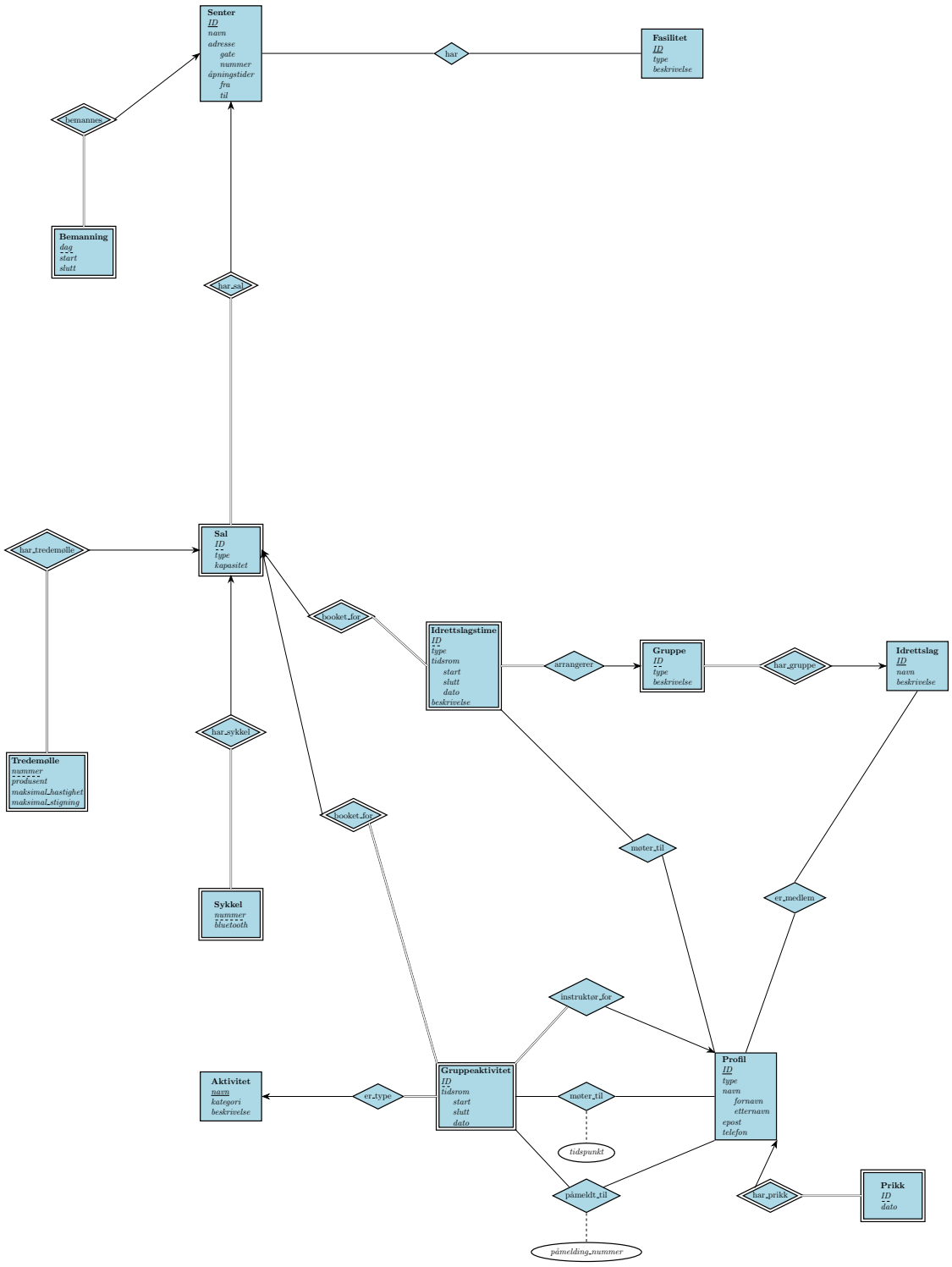

Figure 1: ER-diagram.

1.1 Antakelser

Følgende antakelser er gjort:

1. **Fasiliteter.** Vi har én entitet for fasiliteter med *type* som attributt (f.eks. badstue, garderobe, øvelsesrom). Problembeskrivelsen tolkes slik at det viktige er å beskrive hvilke fasiliteter senteret har, ikke attributtene til hver fasilitetstype. En alternativ modellering ville vært å splitte opp i flere entity sets. Deltagelsen fra Fasilitet i *har*-relasjonen er partiell, slik at en fasilitet kan eksistere i systemet uten å være tilknyttet et senter. Dette forenkler vedlikehold: fasiliteter kan opprettes på forhånd og fjernes fra et senter uten å måtte slettes fra databasen.

2. **Bemanning.** Bemanningstidspunktene antas like fra uke til uke; *dag* er derfor brukt som diskriminator (eng, discriminator). Vi tolker at det viktige er *når* senteret er bemannet, ikke hvem som bemanner. Bemanning kunne alternativt vært koblet til Profil. Et tredje alternativ ville vært å ha bemanning som en attributt for senteret. Ulempen med dette er at vi fort kunne fått null-verdier.

3. **Åpningstider.** Vi antar at et treningssenter har samme åpningstider hver dag.

4. **Plasser på aktivitet.** Antall plasser på en aktivitet er definert ut fra kapasiteten til salen aktiviteten foregår i.

5. **Påmelding og kø.** Køsystemet i påmelding tilrettelegges for med et påmeldingsnummer, som fungerer som kølapp. For en aktivitet i en sal med kapasitet n regnes de n profilene med lavest påmeldingsnummer som påmeldt, resten står på venteliste. For idrettslagstime vil vi bruke et dropin-system, der de n første som møter vil få plass.

6. **Sykler og tredemøller.** Utstyr flyttes ikke mellom rom og identifiseres entydig med nummer og sal.

7. **Instruktør.** Vi har generalisert Profil-entiteten, slik at dette fanger både ansatte/instruktører og vanlige medlemmer. Vi skiller mellom rollene med attributten *type*. Dette kan forsvares med at instruktører potensielt også kan være medlemmer, og dermed delta på andre aktiviteter og være med i idrettslag. I praksis vil typen ansatt funke som et ekstra privilegie der profilen kan være instruktør for en aktivitet, men likevel har alle vanlige medlemsfunksjoner.

8. **Idrettslagstime.** For en undergruppe av et idrettslag som arrangerer idrettslagstime, antas det at en profil ikke trenger å være medlem av gruppen for å delta, altså at det holder at profilen er medlem av idrettslaget.

1.2 Det ER-modellen ikke uttrykker

Følgende regler og restriksjoner kan brukes i et system som bygger på modellen, men uttrykkes ikke eksplisitt i ER-diagrammet:

1. **Prikk ved for sen møting.** En profil får ikke automatisk prikk dersom den møter under 5 minutter før en aktivitet. For å tilrettelegge for prikksystemet skiller vi mellom *påmeldt_til* og *møter_til*, med tidspunkt som attributt i *møter_til*. Da kan man beregne om en profil skal få prikk ved å sjekke om den er påmeldt til en aktivitet uten å møte opp før 5 minutter før start (inkludert ikke møte opp i det hele tatt). Denne regelen er ikke eksplisitt i modellen.

2. **Utestengelse fra nettbooking.** Ved 3 prikker på 30 dager skal profilen utestenges fra nettbooking. Dette kan regnes ut fra en profils prikker og tilhørende datoer. Modellen viser ikke at profilen da ikke kan bruke nettbooking.

3. **Møting etter aktivitetens slutt.** Det skal ikke være mulig å registrere møting til en aktivitet etter aktivitetens slutt. (Vi antar at det er mulig å møte frem til aktivitetens slutt. Det betyr ikke at profilen unngår prikk dersom den møter under 5 minutter før gruppeaktivitetsstart.) Denne restriksjonen vises ikke i ER-modellen.

4. **Ett tidspunkt, én booking per sal.** For et gitt tidspunkt kan en sal bookes av maksimalt én gruppeaktivitet eller én idrettslagstime. Dette er ikke uttrykt i modellen.
5. **Medlemskap for idrettslagstime.** For at en profil skal kunne møte til en idrettslagstime, må den være medlem av idrettslaget. Dette kravet kommer ikke frem i modellen.

2 ER-diagram oversatt til relasjonsskjema

Relasjonsskjemaet under er utledet direkte fra ER-diagrammet ved standardreglene for ER-til-relasjonsoversetting. Primærnøkler er understreket, fremmednøkler er *kursive*, og attributter som er begge deler er *understreket kursive*.

Fasilitet(ID, type, beskrivelse)

Senter(ID, navn, gate, nummer, fra, til)

Idrettslag(ID, navn, beskrivelse)

Profil(ID, type, fornavn, etternavn, epost, telefon)

Aktivitet(navn, kategori, beskrivelse)

har_fasilitet(*senter_ID*, *fasilitet_ID*)

- *senter_ID* → Senter(ID)
- *fasilitet_ID* → Fasilitet(ID)

Bemanning(*senter_ID*, dag, start, slutt)

- *senter_ID* → Senter(ID)

Sal(*senter_ID*, ID, type, kapasitet)

- *senter_ID* → Senter(ID)

Tredemølle(*senter_ID*, *sal_ID*, nummer, produsent, maks_hastighet, maks_stigning)

- (*senter_ID*, *sal_ID*) → Sal(senter_ID, ID)

Sykkel(*senter_ID*, *sal_ID*, nummer, bluetooth)

- (*senter_ID*, *sal_ID*) → Sal(senter_ID, ID)

Gruppe(*idrettslags_ID*, ID, type, beskrivelse)

- *idrettslags_ID* → Idrettslag(ID)

Prikk(*profil_ID*, ID, dato)

- *profil_ID* → Profil(ID)

er_medlem(*idrettslags_ID*, *profil_ID*)

- *idrettslags_ID* → Idrettslag(ID)
- *profil_ID* → Profil(ID)

Idrettslagstime(*senter_ID*, *sal_ID*, ID, type, start, slutt, dato, beskrivelse, *idrettslags_ID*, *gruppe_ID*)

- (*senter_ID*, *sal_ID*) → Sal(senter_ID, ID)
- *idrettslags_ID* → Idrettslag(ID)
- (*idrettslags_ID*, *gruppe_ID*) → Gruppe(idrettslags_ID, ID)

møter_til_idrett(*senter_ID*, *sal_ID*, *idrettslagstime_ID*, *profil_ID*)

- (*senter_ID*, *sal_ID*, *idrettslagstime_ID*) → Idrettslagstime(senter_ID, sal_ID, ID)
- *profil_ID* → Profil(ID)

Gruppeaktivitet(*senter_ID*, *sal_ID*, ID, start, slutt, dato, *aktivitet_navn*, *instrukt_ID*)

- (*senter_ID*, *sal_ID*) → Sal(senter_ID, ID)
- *aktivitet_navn* → Aktivitet(navn)
- *instrukt_ID* → Profil(ID)

møter_til_gruppe(*senter_ID*, *sal_ID*, *gruppeaktivitet_ID*, *profil_ID*, tidspunkt)

- (*senter_ID*, *sal_ID*, *gruppeaktivitet_ID*) → Gruppeaktivitet(senter_ID, sal_ID, ID)
- *profil_ID* → Profil(ID)

påmeldt_til(*senter_ID*, *sal_ID*, *gruppeaktivitet_ID*, *profil_ID*, påmelding_nummer)

- (*senter_ID*, *sal_ID*, *gruppeaktivitet_ID*) → Gruppeaktivitet(senter_ID, sal_ID, ID)
- *profil_ID* → Profil(ID)

2.1 Normalformanalyse

Alle tabellene er på **BCNF** (Boyce–Codd normalform). En relasjon er på BCNF dersom venstre side i enhver ikke-triviell funksjonell avhengighet er en supernøkkel. Nedenfor begrunnes dette gruppevis.

Sterke entiteter – Fasilitet, Senter, Idrettslag, Aktivitet. Disse har én enkeltattributt som primærnøkkel (*ID*, resp. *navn* for Aktivitet). Alle øvrige attributter bestemmes entydig av primærnøkkelen, og det finnes ingen andre funksjonelle avhengigheter. Dermed er venstre side i enhver ikke-triviell FD en supernøkkel, og tabellene er på BCNF.

Profil. Primærnøkkelen er *ID*. I tillegg er *epost* unik for hver profil (brukstilfellene identifiserer brukere ved epostadresse), slik at *epost* → *ID, type, fornavn, etternavn, telefon* også er en ikke-triviell FD. Venstresiden er en kandidatnøkkel og dermed en supernøkkel, så BCNF er oppfylt.

Svake entiteter – Bemanning, Sal, Tredemølle, Sykkel, Idrettslagstime, Gruppeaktivitet, Gruppe, Prikk. ER-til-relasjonsoversettingen av svake entiteter gir en sammensatt primærnøkkel bestående av eierens primærnøkkel og diskriminatoren. Alle ikke-nøkkelattributter avhenger funksjonelt av hele primærnøkkelen og ingen delmengde av den, slik at det ikke finnes partielle avhengigheter. Det finnes heller ingen transitive avhengigheter gjennom ikke-nøkkelattributter. Alle disse tabellene er derfor på BCNF.

M:N-relasjoner uten ekstra attributter – har_fasilitet, er_medlem, møter_til_idrett. Alle attributter inngår i primærnøkkelen. Det finnes ingen ikke-trivielle funksjonelle avhengigheter utover de trivielle, og tabellene er derfor trivielt på BCNF.

møter_til_gruppe. Primærnøkkelen er sammensatt av alle fire fremmednøkkelattributtene. *tidspunkt* avhenger funksjonelt av hele primærnøkkelen, og det finnes ingen andre ikke-trivielle FD-er. Tabellen er på BCNF.

påmeldt_til. Primærnøkkelen er (*senter_ID, sal_ID, gruppeaktivitet_ID, profil_ID*). *påmelding_nummer* er unikt innenfor én gruppeaktivitet (kølapp-semantikk), slik at (*senter_ID, sal_ID, gruppeaktivitet_ID, påmelding_nummer*) → *profil_ID* også er en ikke-triviell FD. Venstresiden er en kandidatnøkkel og dermed en supernøkkel. Begge ikke-trivielle FD-er har altså en supernøkkel på venstresiden, og tabellen er på BCNF.

3 SQL-script

SQL-scriptet under oppretter alle tabellene i databasen med primær- og fremmednøkler samt `CHECK`- og `UNIQUE`-restriksjoner. Scriptet er skrevet for SQLite.

Listing 1: SQL-script for opprettelse av tabeller.

```sql
-- TDT4145 Prosjektoppgave     SQL-script (oppgave c)
-- Oppretter alle tabeller med primær-/fremmednøkler og restriksjoner.
--
-- Målplattform: SQLite  (PRAGMA foreign_keys = ON må kjøres per tilkobling)
--
-- ==========================================================================
-- Restriksjoner som IKKE kan uttrykkes i SQL og må håndteres i applikasjonskode:
--
--   1. Prikk ved manglende/for sen oppmøte: En profil som er påmeldt en
--      gruppeaktivitet uten å registrere oppmøte senest 5 minutter før start
--      skal få en prikk.  Regelen krever tidsberegning og kan ikke fanges av
--      en enkel CHECK-restriksjon.
--
--   2. Utestengelse (svartelisting): Dersom en profil har >= 3 prikker
--      innenfor de siste 30 dagene, skal profilen utestenges fra elektronisk
--      booking inntil den eldste av de tre prikkene er eldre enn 30 dager.
--
--   3. Oppmøte etter aktivitetens slutt: Det skal ikke være mulig å
--      registrere oppmøte til en aktivitet etter at den er ferdig.
--
--   4.  n  booking per sal per tidspunkt: For et gitt tidspunkt kan en sal
--      kun ha  n  gruppeaktivitet eller  n  idrettslagstime.  Overlappsjekk
--      krever intervallsammenligning og kan ikke uttrykkes med UNIQUE alene.
--
--   5. Medlemskap for idrettslagstime: En profil må være medlem av
--      idrettslaget (via er_medlem) for å kunne møte til en idrettslagstime.
--
--   6. Avbestillingsfrist: Avbestilling av en gruppeaktivitet må skje senest
--      1 time før aktivitetens start.
--
--   7. Publiseringsfrist: En gruppeaktivitet legges ut for booking tidligst
--      48 timer før den holdes.
-- ==========================================================================

PRAGMA foreign_keys = ON;

--
-- Sterke entiteter
--

CREATE TABLE Fasilitet (
    ID          INTEGER PRIMARY KEY AUTOINCREMENT,
    type        TEXT    NOT NULL,
    beskrivelse TEXT
);

```

```sql
CREATE TABLE Senter (
    ID     INTEGER PRIMARY KEY AUTOINCREMENT,
    navn   TEXT    NOT NULL UNIQUE,
    gate   TEXT    NOT NULL,
    nummer TEXT    NOT NULL,
    fra    TIME   NOT NULL,
    til    TIME   NOT NULL
);

CREATE TABLE Idrettslag (
    ID          INTEGER PRIMARY KEY AUTOINCREMENT,
    navn        TEXT    NOT NULL,
    beskrivelse TEXT
);

CREATE TABLE Profil (
    ID       INTEGER PRIMARY KEY AUTOINCREMENT,
    type     TEXT    NOT NULL CHECK (type IN ('medlem', 'ansatt')),
    fornavn  TEXT    NOT NULL,
    etternavn TEXT   NOT NULL,
    epost    TEXT    NOT NULL UNIQUE,
    telefon  TEXT    NOT NULL UNIQUE
);

CREATE TABLE Aktivitet (
    navn        TEXT PRIMARY KEY,
    kategori    TEXT NOT NULL,
    beskrivelse TEXT
);

--
-- Svake entiteter og M:N-relasjoner
--

CREATE TABLE har_fasilitet (
    senter_ID    INTEGER NOT NULL,
    fasilitet_ID INTEGER NOT NULL,
    PRIMARY KEY (senter_ID, fasilitet_ID),
    FOREIGN KEY (senter_ID)    REFERENCES Senter(ID),
    FOREIGN KEY (fasilitet_ID) REFERENCES Fasilitet(ID)
);

CREATE TABLE Bemanning (
    senter_ID INTEGER NOT NULL,
    dag       TEXT    NOT NULL CHECK (dag IN (
                  'mandag','tirsdag','onsdag','torsdag',
                  'fredag','lørdag','søndag')),
    start     TIME    NOT NULL,
    slutt     TIME    NOT NULL,
    PRIMARY KEY (senter_ID, dag),
    FOREIGN KEY (senter_ID) REFERENCES Senter(ID),
    CHECK (start < slutt)
);

CREATE TABLE Sal (
    senter_ID INTEGER NOT NULL,
    ID        INTEGER NOT NULL,
    type      TEXT    NOT NULL,
    kapasitet INTEGER NOT NULL CHECK (kapasitet > 0),
    PRIMARY KEY (senter_ID, ID),
    FOREIGN KEY (senter_ID) REFERENCES Senter(ID)
);

CREATE TABLE Tredemølle (
    senter_ID      INTEGER NOT NULL,
    sal_ID         INTEGER NOT NULL,
    nummer         INTEGER NOT NULL,
    produsent      TEXT    NOT NULL,
    maks_hastighet REAL    NOT NULL CHECK (maks_hastighet > 0),
```

```sql
    maks_stigning  REAL    NOT NULL CHECK (maks_stigning  > 0),
    PRIMARY KEY (senter_ID, sal_ID, nummer),
    FOREIGN KEY (senter_ID, sal_ID) REFERENCES Sal(senter_ID, ID)
);

CREATE TABLE Sykkel (
    senter_ID INTEGER NOT NULL,
    sal_ID    INTEGER NOT NULL,
    nummer    INTEGER NOT NULL,
    bluetooth INTEGER NOT NULL DEFAULT 0 CHECK (bluetooth IN (0, 1)),
    PRIMARY KEY (senter_ID, sal_ID, nummer),
    FOREIGN KEY (senter_ID, sal_ID) REFERENCES Sal(senter_ID, ID)
);

CREATE TABLE Gruppe (
    idrettslags_ID INTEGER NOT NULL,
    ID             INTEGER NOT NULL,
    type           TEXT    NOT NULL,
    beskrivelse    TEXT,
    PRIMARY KEY (idrettslags_ID, ID),
    FOREIGN KEY (idrettslags_ID) REFERENCES Idrettslag(ID)
);

CREATE TABLE Prikk (
    profil_ID INTEGER NOT NULL,
    ID        INTEGER NOT NULL,
    dato      DATE    NOT NULL,
    PRIMARY KEY (profil_ID, ID),
    FOREIGN KEY (profil_ID) REFERENCES Profil(ID)
);

CREATE TABLE er_medlem (
    idrettslags_ID INTEGER NOT NULL,
    profil_ID      INTEGER NOT NULL,
    PRIMARY KEY (idrettslags_ID, profil_ID),
    FOREIGN KEY (idrettslags_ID) REFERENCES Idrettslag(ID),
    FOREIGN KEY (profil_ID)      REFERENCES Profil(ID)
);

CREATE TABLE Idrettslagstime (
    senter_ID      INTEGER NOT NULL,
    sal_ID         INTEGER NOT NULL,
    ID             INTEGER NOT NULL,
    type           TEXT    NOT NULL,
    start          TIME    NOT NULL,
    slutt          TIME    NOT NULL,
    dato           DATE    NOT NULL,
    beskrivelse    TEXT,
    idrettslags_ID INTEGER NOT NULL,
    gruppe_ID      INTEGER NOT NULL,
    PRIMARY KEY (senter_ID, sal_ID, ID),
    FOREIGN KEY (senter_ID, sal_ID)            REFERENCES Sal(senter_ID, ID),
    FOREIGN KEY (idrettslags_ID)               REFERENCES Idrettslag(ID),
    FOREIGN KEY (idrettslags_ID, gruppe_ID)    REFERENCES Gruppe(idrettslags_ID, ID
        ),
    CHECK (start < slutt)
);

CREATE TABLE møter_til_idrett (
    senter_ID          INTEGER NOT NULL,
    sal_ID             INTEGER NOT NULL,
    idrettslagstime_ID INTEGER NOT NULL,
    profil_ID          INTEGER NOT NULL,
    PRIMARY KEY (senter_ID, sal_ID, idrettslagstime_ID, profil_ID),
    FOREIGN KEY (senter_ID, sal_ID, idrettslagstime_ID)
        REFERENCES Idrettslagstime(senter_ID, sal_ID, ID),
    FOREIGN KEY (profil_ID) REFERENCES Profil(ID)
);

CREATE TABLE Gruppeaktivitet (
    senter_ID      INTEGER NOT NULL,
    sal_ID         INTEGER NOT NULL,
    ID             INTEGER NOT NULL,
```

```sql
    start           TIME    NOT NULL,
    slutt           TIME    NOT NULL,
    dato            DATE    NOT NULL,
    aktivitet_navn  TEXT    NOT NULL,
    instrukt_ID     INTEGER NOT NULL,
    PRIMARY KEY (senter_ID, sal_ID, ID),
    FOREIGN KEY (senter_ID, sal_ID) REFERENCES Sal(senter_ID, ID),
    FOREIGN KEY (aktivitet_navn)    REFERENCES Aktivitet(navn),
    FOREIGN KEY (instrukt_ID)       REFERENCES Profil(ID),
    CHECK (start < slutt)
);

CREATE TABLE møter_til_gruppe (
    senter_ID           INTEGER NOT NULL,
    sal_ID              INTEGER NOT NULL,
    gruppeaktivitet_ID  INTEGER NOT NULL,
    profil_ID           INTEGER NOT NULL,
    tidspunkt           DATETIME NOT NULL,
    PRIMARY KEY (senter_ID, sal_ID, gruppeaktivitet_ID, profil_ID),
    FOREIGN KEY (senter_ID, sal_ID, gruppeaktivitet_ID)
        REFERENCES Gruppeaktivitet(senter_ID, sal_ID, ID),
    FOREIGN KEY (profil_ID) REFERENCES Profil(ID)
);

CREATE TABLE påmeldt_til (
    senter_ID           INTEGER NOT NULL,
    sal_ID              INTEGER NOT NULL,
    gruppeaktivitet_ID  INTEGER NOT NULL,
    profil_ID           INTEGER NOT NULL,
    påmelding_nummer    INTEGER NOT NULL,
    PRIMARY KEY (senter_ID, sal_ID, gruppeaktivitet_ID, profil_ID),
    UNIQUE (senter_ID, sal_ID, gruppeaktivitet_ID, påmelding_nummer),
    FOREIGN KEY (senter_ID, sal_ID, gruppeaktivitet_ID)
        REFERENCES Gruppeaktivitet(senter_ID, sal_ID, ID),
    FOREIGN KEY (profil_ID) REFERENCES Profil(ID)
);
```

3.1 Restriksjoner håndtert i applikasjonskode

Følgende restriksjoner kan ikke uttrykkes med `CREATE TABLE`-definisjoner alene og må derfor håndheves i applikasjonsprogrammene:

1. **Prikk ved manglende/for sen oppmøte.** En profil som er påmeldt en gruppeaktivitet uten å registrere oppmøte senest 5 minutter før start, skal få en prikk. Regelen krever tidsberegning og kan ikke fanges av en enkel `CHECK`-restriksjon.

2. **Utestengelse (svartelisting).** Dersom en profil har ≥ 3 prikker innenfor de siste 30 dagene, skal profilen utestenges fra elektronisk booking inntil den eldste av de tre prikkene er eldre enn 30 dager.

3. **Oppmøte etter aktivitetens slutt.** Det skal ikke være mulig å registrere oppmøte til en aktivitet etter at den er ferdig.

4. **Én booking per sal per tidspunkt.** For et gitt tidspunkt kan en sal kun ha én gruppeaktivitet eller én idrettslagstime. Overlappsjekk krever intervallsammenligning og kan ikke uttrykkes med `UNIQUE` alene.

5. **Medlemskap for idrettslagstime.** En profil må være medlem av idrettslaget (via `er_medlem`) for å kunne møte til en idrettslagstime.

6. **Avbestillingsfrist.** Avbestilling av en gruppeaktivitet må skje senest 1 time før aktivitetens start.

7. **Publiseringsfrist.** En gruppeaktivitet legges ut for booking tidligst 48 timer før den holdes.

4 Svar på spørsmål